

LAPORAN PRAKTIKUM
STRUKTUR DATA
SEARCHING (BFS DAN DFS)

Disusun Oleh :

Nama : Nabila Khairunnisa
Nim : 2511531003
Dosen Pengampu : Dr. Wahyudi, S.T, M.T
Asisten Praktikum : M. Galid Avero



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji Syukur atas kehadiran Allah SWT yang telah melimpahkan Rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum struktur data dengan judul “Sorting 2” dengan baik dan tepat waktu. Dalam menyelesaikan laporan ini saya banyak mendapat arahan dan bimbingan, oleh karena itu saya ingin mengucapkan terimakasih kepada

1. Bapak Dr. Wahyudi, S.T, M.T selaku dosen pengampu
2. Uda M. Galid Averro selaku asisten labor
3. Orang tua yang senantiasa mendoakan
4. Teman teman yang selalu memotivasi

Penulis menyadari bahwa laporan ini jauh dari kata sempurna. Oleh sebab itu, penulis sangat membuka diri apabila ada yang ingin memberikan kritikan dan saran yang sifatnya membantu, penulis akan sangat senang menerima. Tujuannya agar untuk kedepannya bisa menyempurnakan laporan.

Padang, 8 Juni 2026

Nabila Khairunnisa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN.....	3
2.1 Deskripsi Praktikum.....	3
2.2 Langkah Langkah Praktikum.....	3
BAB III KESIMPULAN.....	10
3.1 Kesimpulan	10
3.2 Saran	10
DAFTAR PUSTAKA	11
LAPORAN TUGAS PEKAN 9	12

BAB I

PENDAHULUAN

1.1 Latar Belakang

BFS adalah algoritma yang digunakan untuk membuat grafik data atau menelusuri pohon atau struktur yang melintasinya. Algoritma ini secara efisien mengunjungi dan menandai semua simpul kunci dalam grafik dengan cara yang akurat. Algoritma ini memilih satu simpul (titik awal atau titik sumber) dalam grafik, lalu mengunjungi semua simpul yang berdekatan dengan simpul yang dipilih. Setelah algoritma mengunjungi dan menandai simpul awal, ia bergerak menuju simpul terdekat yang belum dikunjungi dan menganalisisnya.

DFS adalah algoritma untuk menemukan atau menelusuri graf atau pohon ke arah kedalaman. Eksekusi algoritma dimulai dari simpul akar dan menjelajahi setiap cabang sebelum kembali. Algoritma ini menggunakan struktur data tumpukan (stack) untuk mengingat, mendapatkan simpul berikutnya, dan memulai pencarian setiap kali menemui jalan buntu dalam iterasi apa pun. Kepanjangan dari DFS adalah Depth-first search (Pencarian Mendalam Pertama). [1]

Kelebihan dari DFS yaitu Membutuhkan memori yang relatif kecil, karena hanya node-node pada lintasan yang aktif saja yang disimpan. Namun DFS juga memiliki kekurangan yaitu Jika terdapat lebih dari 1 solusi yang sama tetapi berada pada level yang berbeda, maka pada DFS tidak ada jaminan untuk menemukan solusi yang paling baik (Tidak Optimal).

Algoritma Depth First Search adalah algoritma pencarian mendalam yang dimulai dari node awal dilanjutkan dengan hanya mengunjungi node anak paling kiri pada tingkat selanjutnya. [2]

Dalam algoritma BFS, simpul anak yang telah dikunjungi disimpan dalam suatu antrian. Antrian ini digunakan untuk mengacu simpul-simpul yang bertetangga dengannya yang akan dikunjungi kemudian sesuai urutan pengantrian. [3]

1.2 Tujuan Praktikum

1. Mampu memahami konsep Tree dan Graph
2. Mampu membuat dan mengimplementasikan Binary Tree
3. Mampu memahami hubungan antar node
4. Mampu memahami Teknik traversal pada Binary Tree menggunakan metode InOrder, PreOrder, dan PostOrder

1.3 Manfaat Praktikum

1. Mahasiswa mampu memahami konsep Tree dan Graph
2. Mahasiswa mampu membuat dan mengimplementasikan Binary Tree
3. Mahasiswa mampu memahami hubungan antar node
4. Mahasiswa mampu memahami Teknik traversal pada Binary Tree menggunakan metode InOrder, PreOrder, dan PostOrder

BAB II

PEMBAHASAN

2.1 Deskripsi Praktikum

Praktikum Struktur data pada pekan ini adalah implementasi struktur data Tree dan Graph menggunakan bahasa pemrograman Java. Praktikum diawali dengan pembuatan class Node yang berfungsi sebagai representasi setiap simpul pada struktur pohon, yang menyimpan data serta referensi ke anak kiri dan anak kanan. Selanjutnya, class BTree (Binary Tree) digunakan untuk membangun struktur pohon biner dan melakukan proses traversal menggunakan metode PreOrder, InOrder, dan PostOrder. Untuk menjalankan dan menguji implementasi Binary Tree, digunakan class BTreeDriver sebagai class utama yang membuat objek pohon, memasukkan data, dan menampilkan hasil traversal. Selain itu, praktikum juga membahas class GraphTraversal yang digunakan untuk melakukan penelusuran pada struktur data Graph menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS).

2.2 Langkah Langkah Praktikum

- 1) Buatlah package terlebih dahulu dengan mengklik kanan di folder PrakSD2026_C_2511531003/src , pilih new dan klik package. Setelah itu beri nama pada package tanpa huruf kapital, karakter khusus serta tanpa “space”. lalu “finish”.
- 2) Klik kanan pada package pekan9_2511531003 yang sudah dibuat sebelumnya, pilih “New”, lalu pilih class. Buat nama “Node_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan untuk erepresentasikan satu node pada Binary Tree. Setiap node menyimpan data dan memiliki dua cabang, yaitu left dan right.
- 3) Tuliskan program seperti berikut.

```

1 package pekar9_2511531003;
2
3 public class Node_2511531003 {
4     int data;
5     Node_2511531003 left;
6     Node_2511531003 right;
7     public Node_2511531003(int data) {
8         this.data = data;
9         left = null;
10        right = null;
11    }
12    public void setLeft_1003 (Node_2511531003 node) {
13        if (left == null)
14            left = node;
15    }
16    public void setRight_1003 (Node_2511531003 node) {
17        if (right == null)
18            right = node;
19    }
20    public Node_2511531003 getLeft_1003 () {
21        return left;
22    }
23    public Node_2511531003 getRight_1003 () {
24        return right;
25    }
26    public int getData_1003 () {
27        return data;
28    }
29    public void setData_1003 (int data) {
30        this.data = data;
31    }
32
33    void printPreorder_1003 (Node_2511531003 node) {
34        if (node == null)
35            return;
36        System.out.print (node.data + " ");
37        printPreorder_1003 (node.left);
38        printPreorder_1003 (node.right);
39    }
40    void printPostorder_1003 (Node_2511531003 node) {
41        if (node == null)
42            return;
43        printPostorder_1003 (node.left);
44        printPostorder_1003 (node.right);
45        System.out.print (node.data + " ");
46    }
47    void printInorder_1003 (Node_2511531003 node) {
48        if (node == null)
49            return;
50        printInorder_1003 (node.left);
51        System.out.print (node.data + "-> ");
52        printInorder_1003 (node.right);
53    }
54    public String print_1003 () {
55        return this.print("",true,"");
56    }
57    public String print (String prefix, boolean isTail, String sb) {
58        if (right != null) {
59            right.print (prefix + (isTail ? "| " : " "), false, sb);
60        }
61        System.out.println(
62            prefix + (isTail ? "\\-- " : "/-- ") + data
63        );
64        if (left != null) {
65            left.print (prefix + (isTail ? " " : "| "), true, sb);
66        }
67        return sb;
68    }
69 }

```

Gambar 2.2.1 kode program

- *Int data* untuk menyimpan nilai yang terdapat pada node.
- *Left* untuk menyimpan alamat anak kiri.
- *Right* untuk menyimpan alamat anak kanan.
- *Node_2511531003(int data)* merupakan bagian constuctor yang digunakan untuk membuat node baru.
- *setLeft_1003* untuk menambahkan anak kiri pada node.
- *setRight_1003* untuk menambahkan anak kanan pada node.
- *getLeft_1003* untuk mengambil node anak kiri.
- *getRight_1003* untuk mengambil node anak kanan.
- *getData_1003* untuk mengambil nilai data yang tersimpan pada node.
- *setData_1003* untuk mengubah nilai data pada node.
- *void printPreorder_1003* untuk menampilkan data dengan traversal Preorder
- *void printPostorder_1003* untuk Menampilkan data dengan traversal PostOrder.
- *void printInorder_1003* untuk Menampilkan data dengan traversal InOrder.
- *print_1003* untuk menampilkan struktur tree secara keseluruhan.
- *Print()* untuk Menampilkan bentuk Binary Tree secara visual.

- 4) Buat lah class kedua dengan nama “BTree_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini digunakan untuk menyimpan data dalam bentuk pohon biner serta melakukan operasi pencarian, percetakan traversal, dan menghitung jumlah node.
- 5) Tuliskan program seperti berikut.

```

1 package pekan9_2511531003;
2
3 public class BTree_2511531003 {
4     private Node_2511531003 root_1003;
5     private Node_2511531003 currentNode_1003;
6     public BTree_2511531003() {
7         root_1003 = null;
8     }
9     public boolean search_1003(int data_1003) {
10        return search_1003(root_1003, data_1003);
11    }
12    private boolean search_1003(Node_2511531003 node_1003, int data_1003) {
13        if (node_1003.getData_1003() == data_1003)
14            return true;
15        if (node_1003.getleft_1003() != null)
16            return search_1003(node_1003.getleft_1003(), data_1003);
17        if (node_1003.getright_1003() != null)
18            return search_1003(node_1003.getright_1003(), data_1003);
19        return false;
20    }
21    public void printInorder_1003() {
22        root_1003.printInorder_1003(root_1003);
23    }
24    public void printPreorder_1003() {
25        root_1003.printPreorder_1003(root_1003);
26    }
27    public void printPostorder_1003() {
28        root_1003.printPostorder_1003(root_1003);
29    }
30    public Node_2511531003 getRoot_1003() {
31        return root_1003;
32    }
33    public boolean isEmpty_1003() {
34        return root_1003 == null;
35    }
36
37    public int countNodes_1003() {
38        return countNodes_1003(root_1003);
39    }
40    private int countNodes_1003(Node_2511531003 node_1003) {
41        int count_1003 = 1;
42        if (node_1003 == null) {
43            return 0;
44        } else {
45            count_1003 += countNodes_1003(node_1003.getleft_1003());
46            count_1003 += countNodes_1003(node_1003.getright_1003());
47            return count_1003;
48        }
49    }
50    public void print_1003() {
51        root_1003.print_1003();
52    }
53    public Node_2511531003 getCurrent_1003() {
54        return currentNode_1003;
55    }
56    public void setCurrent_1003(Node_2511531003 node_1003) {
57        this.currentNode_1003 = node_1003;
58    }
59    public void setRoot_1003(Node_2511531003 root_1003) {
60        this.root_1003 = root_1003;
61    }
62
63 }

```

Gambar 2.2.2 Kode Program

- *Root_1003* untuk menyimpan node pertama (akar) dari tree.
- *currentNode_1003* untuk menyimpan node yang sedang aktif atau sedang di proses.
- *Public BTree_2511531003* merupakan bagian constructor yang berfungsi untuk membuat tree baru dan nilai root diisi null karena tree masih kosong.
- *Search_1003* untuk mencari data tertentu pada tree.
- *private boolean search_1003(Node_2511531003 node_1003, int data_1003)* untuk membandingkan data node dengan data yang dicari.
- *printInorder_1003* untuk menampilkan data dengan urutan kiri → root → kanan.
- *printPreorder_1003* untuk menampilkan data dengan urutan root → kiri → kanan.
- *printPostorder_1003* untuk menampilkan data dengan urutan kiri → kanan → root.

- *getRoot_1003* untuk mengembalikan node akar (root).
 - *isEmpty_1003* untuk memeriksa apakah tree kosong.
 - *countNodes_1003* untuk memanggil method rekursif untuk menghitung node.
 - *countNodes_1003(Node_2511531003 node_1003)* untuk Menghitung seluruh node dalam tree.
 - *print_1003* untuk menampilkan seluruh isi tree yang ada pada class *Node_2511531003*.
 - *getCurrent_1003* untuk mengambil node yang sedang aktif.
 - *setCurrent_1003* untuk mengubah node yang sedang aktif.
 - *setRoot_1003* untuk mengganti node akar (root) tree.
- 6) Class ketiga dengan nama “BTreeDriver_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi untuk menjalankan program Binary Tree. Di dalam class ini dilakukan pembuatan tree, penambahan node, perhitungan jumlah node, traversal tree, dan menampilkan bentuk pohon.
- 7) Tuliskan program seperti berikut.

```

1 package pekan9_2511531003;
2
3 public class BTreeDriver_2511531003 {
4     public static void main(String[] args) {
5         // membuat pohon
6         BTree_2511531003 tree_1003 = new BTree_2511531003();
7         System.out.println("Jumlah simpul awal pohon: ");
8         System.out.println(tree_1003.countNodes_1003());
9         // menambahkan simpul data 1
10        Node_2511531003 root_1003 = new Node_2511531003(1);
11        // menjadikan simpul 1 sebagai root
12        tree_1003.setRoot_1003(root_1003);
13        System.out.println("Jumlah simpul jika hanya ada root");
14        System.out.println(tree_1003.countNodes_1003());
15        Node_2511531003 node2_1003 = new Node_2511531003(2);
16        Node_2511531003 node3_1003 = new Node_2511531003(3);
17        Node_2511531003 node4_1003 = new Node_2511531003(4);
18        Node_2511531003 node5_1003 = new Node_2511531003(5);
19        Node_2511531003 node6_1003 = new Node_2511531003(6);
20        Node_2511531003 node7_1003 = new Node_2511531003(7);
21        Node_2511531003 node8_1003 = new Node_2511531003(8);
22        Node_2511531003 node9_1003 = new Node_2511531003(9);
23        root_1003.setLeft_1003(node2_1003);
24        node2_1003.setLeft_1003(node4_1003);
25        node2_1003.setRight_1003(node5_1003);
26        node4_1003.setRight_1003(node6_1003);
27        root_1003.setRight_1003(node3_1003);
28        node3_1003.setLeft_1003(node6_1003);
29        node3_1003.setRight_1003(node7_1003);
30        node6_1003.setLeft_1003(node9_1003);
31        // set root 1003
32        tree_1003.setCurrent_1003(tree_1003.getRoot_1003());
33        System.out.println("menampilkan simpul terakhir: ");
34        System.out.println(tree_1003.getCurrent_1003().getdata_1003());
35        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
36        System.out.println(tree_1003.countNodes_1003());
37        System.out.println("InOrder: ");
38        tree_1003.printInOrder_1003();
39        System.out.println("PreOrder: ");
40        tree_1003.printPreOrder_1003();
41        System.out.println("PostOrder: ");
42        tree_1003.printPostOrder_1003();
43        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
44        tree_1003.print_1003();
45    }
46 }
47 }

```

Gambar 2.2.3 Kode Program

- *BTree_2511531003 tree_1003 = new BTree_2511531003()* untuk membuat objek binary tree yang masih kosong.
- *Node_2511531003 root_1003 = new Node_2511531003(1)* untuk membuat node dengan nilai 1.
- *tree_1003.setRoot_1003* untuk menjadikan node 1 menjadi akar pohon.
- *Node_2511531003 node2_1003 = new Node_2511531003(2)* untuk membuat node lain (2)

- `root_1003.setLeft_1003(node2_1003)` untuk menyusun struktur binary tree.
- `tree_1003.setCurrent_1003(tree_1003.getRoot_1003())` untuk menentukan current node.
- `tree_1003.printInorder_1003` untuk mengurutkan kiri → root → kanan.
- `tree_1003.printPreorder_1003` untuk mengurutkan root → kiri → kanan.
- `tree_1003.printPostorder_1003` untuk mengurutkan kiri → kanan → root.
- `tree_1003.print_1003` untuk menampilkan struktur tree secara visual.

8) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> BTreeDriver_2511531003 [Java Application] C:\Users\lenovo\p2\pool\p
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|      /-- 7
|      /-- 3
|      |      \-- 6
|      |      \-- 9
|      |      \-- 1
|      |      /-- 5
|      |      \-- 2
|      |      |      /-- 8
|      |      |      \-- 4
```

Gambar 2.2.4 Output Program

9) Class ke empat dengan nama “GraphTraversal_2511531003” dengan ketentuan nama harus Uppercase pada awal kalimat dan tanpa “spasi”, lalu “finish”. Class ini berfungsi untuk membuat dan menelusuri graf (graph) menggunakan dua algoritma traversal.

10) Tuliskan program seperti berikut

```

1 //GraphTraversal_2511231003
2 import java.util.*;
3 public class GraphTraversal_2511231003 {
4     private Map<String, List<String>> graph_1003 = new HashMap<>();
5     // Hamiltonian and DFS
6     public void addEdge_1003(String node1_1003, String node2_1003) {
7         graph_1003.putIfAbsent(node1_1003, new ArrayList<>());
8         graph_1003.putIfAbsent(node2_1003, new ArrayList<>());
9         graph_1003.get(node1_1003).add(node2_1003);
10        graph_1003.get(node2_1003).add(node1_1003);
11    }
12    // Hamiltonian and DFS
13    public void printGraph_1003() {
14        System.out.println("Graph Adjacency List:");
15        for (String node_1003 : graph_1003.keySet()) {
16            System.out.print(node_1003 + " : ");
17            List<String> neighbors_1003 = graph_1003.get(node_1003);
18            System.out.println(neighbors_1003);
19        }
20        System.out.println();
21    }
22    // DFS Helper
23    public void dfs_1003(String start_1003) {
24        Set<String> visited_1003 = new HashSet<>();
25        dfsHelper_1003(start_1003, visited_1003);
26    }
27    private void dfsHelper_1003(String current_1003, Set<String> visited_1003) {
28        if (visited_1003.contains(current_1003))
29            return;
30        visited_1003.add(current_1003);
31        System.out.print(current_1003 + " ");
32        for (String neighbor_1003 : graph_1003.getOrDefault(current_1003, new ArrayList<>())) {
33            dfsHelper_1003(neighbor_1003, visited_1003);
34        }
35    }
36    // BFS Helper
37    public void bfs_1003(String start) {
38        Set<String> visited = new HashSet<>();
39        Queue<String> queue = new LinkedList<>();
40        queue.add(start);
41        visited.add(start);
42        while (!queue.isEmpty()) {
43            String current = queue.poll();
44            System.out.print(current + " ");
45            for (String neighbor : graph_1003.getOrDefault(current, new ArrayList<>())) {
46                if (!visited.contains(neighbor)) {
47                    queue.add(neighbor);
48                    visited.add(neighbor);
49                }
50            }
51        }
52        System.out.println();
53    }
54    // Main
55    public static void main(String[] args) {
56        GraphTraversal_2511231003 graph_1003 = new GraphTraversal_2511231003();
57        graph_1003.addEdge_1003("A", "B");
58        graph_1003.addEdge_1003("B", "C");
59        graph_1003.addEdge_1003("C", "A");
60        graph_1003.addEdge_1003("A", "D");
61        graph_1003.addEdge_1003("D", "A");
62        System.out.println("Graph Adjacency List:");
63        graph_1003.printGraph_1003();
64        graph_1003.dfs_1003("A");
65        graph_1003.bfs_1003("A");
66    }
67 }

```

Gambar 2.2.5 Kode Program

- *Map<String, List<String>> graph_1003 = new HashMap<>()* untuk menyimpan graf
- *addEdge_1003* untuk menambahkan hubungan (edge) antara dua simpul.
- *graph_1003.putIfAbsent(node1_1003, new ArrayList<>())* untuk membuat simpul jika belum ada.
- *graph_1003.get(node1_1003).add(node2_1003)* karena graf tidak berarah, maka hubungan dibuat dua arah.
- *printGraph_1003* untuk menampilkan isi graf dalam bentuk Adjacency List.
- *dfs_1003* untuk Menjalankan algoritma Depth First Search (DFS).
- *Set<String> visited_1003 = new HashSet<>()* untuk menyimpan simpul yang sudah dikunjungi agar tidak terjadi pengulangan.
- *dfsHelper_1003* untuk memanggil DFS Rekursif
- *bfs_1003* untuk melakukan traversal menggunakan algoritma BFS.
- *while (!queue.isEmpty())* untuk perulangan BFS.

- 11) Jalankan program dengan menekan tombol hijau bergambar ► (Run) pada kiri atas di Menu Bar. Kemudian Program akan menghasilkan output seperti berikut

```
<terminated> GraphTraversal_2511531003 [Java Applic
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

Gambar 2.2.6 Output Program

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Tree dan Graph merupakan struktur data non-linear yang digunakan untuk menyimpan dan mengelola data yang memiliki hubungan antar elemen. Pada praktikum ini telah berhasil dibuat **Binary Tree** yang terdiri dari node root, right, dan left serta dilakukan traversal menggunakan metode InOrder, PreOrder, dan PostOrder untuk menelusuri seluruh node pada pohon. Selain itu, telah berhasil diimplementasikan Graph Traversal menggunakan representasi Adjacency List dengan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). DFS melakukan penelusuran hingga ke cabang terdalam terlebih dahulu, sedangkan BFS menelusuri simpul berdasarkan level. Dari hasil praktikum dapat dipahami cara kerja, implementasi, serta perbedaan karakteristik traversal pada struktur data Tree dan Graph.

3.2 Saran

Untuk menambah pengembangan praktikum di masa mendatang disarankan untuk memahami konsep node, edge, root, parent, child, dan traversal. Perbanyak Latihan menggunakan algoritma DFS dan BFS karena algoritma tersebut sering digunakan dalam berbagai kasus pemrograman dan pengembangan sistem.

DAFTAR PUSTAKA

- [1] S. Chen, "BFS vs DFS – Perbedaan Antara Keduanya," GURU99, 26 September 2024. [Online]. Available: <https://www.guru99.com/id/difference-between-bfs-and-dfs.html>. [Accessed 9 Juni 2026].
- [2] D. Tamara, "DFS (Depth First Search) : Pengertian, Kekurangan, Kelebihan, dan Contohnya," Medium, 16 Oktober 2021. [Online]. Available: <https://medium.com/@defytamara2610/dfs-depth-first-search-pengertian-kekurangan-kelebihan-dan-contohnya-2b9b1eee2b3d>. [Accessed 10 Juni 2026].
- [3] michaeljulius, "Pengertian Metode Pencarian BFS dan DFS, beserta Contoh nya," michaeljulius11, 21 Oktober 2017. [Online]. Available: <https://michaeljulius11.blogspot.com/2017/10/pengertian-metode-pencarian-bfs-dan-dfs.html>. [Accessed 10 Juni 2026].

LAPORAN TUGAS PRAKTIKUM PEKAN 9

1. Source Code Java

```
1. package pekan9_2511531003;
2.
3. import javax.swing.*;
4. import java.awt.*;
5. import java.util.*;
6. import java.util.List;
7.
8. public class PetaBandara_2511531003 extends JFrame {
9.
10.     private Map<String, List<String>> graph_1003 = new HashMap<>();
11.
12.     private JComboBox<String> start_1003;
13.     private JComboBox<String> goal_1003;
14.
15.     private JTextArea hasil_1003;
16.
17.     private GraphPanel_1003 panelGraph_1003;
18.
19.     public PetaBandara_2511531003() {
20.
21.         setTitle("Peta Bandara BFS DFS - 2511531003");
22.         setSize(1000,700);
23.         setDefaultCloseOperation(EXIT_ON_CLOSE);
24.         setLocationRelativeTo(null);
25.
26.         buatGraph_1003();
27.
28.         JPanel atas = new JPanel();
29.
30.         start_1003 = new JComboBox<>(
31.             graph_1003.keySet().toArray(new String[0]));
32.
33.         goal_1003 = new JComboBox<>(
34.             graph_1003.keySet().toArray(new String[0]));
35.
36.         JButton bfsBtn = new JButton("BFS");
37.         JButton dfsBtn = new JButton("DFS");
38.         JButton resetBtn = new JButton("RESET");
39.
40.         atas.add(new JLabel("Start"));
41.         atas.add(start_1003);
42.
43.         atas.add(new JLabel("Goal"));
44.         atas.add(goal_1003);
45.
46.         atas.add(bfsBtn);
47.         atas.add(dfsBtn);
48.         atas.add(resetBtn);
49.
50.         panelGraph_1003 = new GraphPanel_1003();
51.
52.         hasil_1003 = new JTextArea();
53.         hasil_1003.setEditable(false);
54.
55.         JSplitPane split = new JSplitPane(
56.             JSplitPane.HORIZONTAL_SPLIT,
57.             panelGraph_1003,
58.             new JScrollPane(hasil_1003));
59.
60.         split.setDividerLocation(650);
61.
62.         add(atas, BorderLayout.NORTH);
```

```

63.         add(split, BorderLayout.CENTER);
64.
65.         bfsBtn.addActionListener(e -> BFS_1003());
66.
67.         dfsBtn.addActionListener(e -> DFS_1003());
68.
69.         resetBtn.addActionListener(e -> resetGraph_1003());
70.     }
71.
72.     private void addEdge_1003(String a, String b){
73.
74.         graph_1003.putIfAbsent(a,new ArrayList<>());
75.         graph_1003.putIfAbsent(b,new ArrayList<>());
76.
77.         graph_1003.get(a).add(b);
78.         graph_1003.get(b).add(a);
79.     }
80.
81.     private void buatGraph_1003(){
82.
83.         addEdge_1003("Pintu Masuk","Check In");
84.         addEdge_1003("Pintu Masuk","Bagasi");
85.
86.         addEdge_1003("Check In","Security");
87.         addEdge_1003("Check In","Ruang Tunggu");
88.
89.         addEdge_1003("Security","Terminal A");
90.         addEdge_1003("Security","Terminal B");
91.
92.         addEdge_1003("Ruang Tunggu","Terminal A");
93.         addEdge_1003("Ruang Tunggu","Terminal B");
94.
95.         addEdge_1003("Terminal A","Gate 1");
96.         addEdge_1003("Terminal A","Gate 2");
97.
98.         addEdge_1003("Terminal B","Gate 2");
99.         addEdge_1003("Terminal B","Gate 3");
100.
101.         addEdge_1003("Gate 1","Bagasi");
102.         addEdge_1003("Gate 2","Bagasi");
103.         addEdge_1003("Gate 3","Bagasi");
104.     }
105.
106.     private void BFS_1003(){
107.
108.         String start = start_1003.getSelectedItemAt().toString();
109.         String goal = goal_1003.getSelectedItemAt().toString();
110.
111.         Queue<String> queue = new LinkedList<>();
112.         Set<String> visited = new LinkedHashSet<>();
113.         Map<String,String> parent = new HashMap<>();
114.
115.         queue.add(start);
116.         visited.add(start);
117.
118.         while(!queue.isEmpty()){
119.
120.             String current = queue.poll();
121.
122.             if(current.equals(goal))
123.                 break;
124.
125.             for(String next :
126.                 graph_1003.getDefault(current,new ArrayList<>())){
127.
128.                 if(!visited.contains(next)){
129.
130.                     visited.add(next);
131.                     parent.put(next,current);

```



```

132.         queue.add(next);
133.     }
134. }
135.
136.
137.     tampilkanHasil_1003(
138.         "BFS",
139.         visited,
140.         parent,
141.         start,
142.         goal);
143.
144.     panelGraph_1003.setVisited(visited);
145. }
146.
147. private void DFS_1003(){
148.
149.     String start = start_1003.getSelectedItemAt().toString();
150.     String goal = goal_1003.getSelectedItemAt().toString();
151.
152.     Stack<String> stack = new Stack<>();
153.
154.     Set<String> visited = new LinkedHashSet<>();
155.
156.     Map<String,String> parent = new HashMap<>();
157.
158.     stack.push(start);
159.
160.     while(!stack.isEmpty()){
161.
162.         String current = stack.pop();
163.
164.         if(!visited.contains(current)){
165.
166.             visited.add(current);
167.
168.             if(current.equals(goal))
169.                 break;
170.
171.             List<String> tetangga =
172.                 graph_1003.get(current);
173.
174.             for(String n : tetangga){
175.
176.                 if(!visited.contains(n)){
177.
178.                     parent.put(n,current);
179.                     stack.push(n);
180.                 }
181.             }
182.         }
183.     }
184.
185.     tampilkanHasil_1003(
186.         "DFS",
187.         visited,
188.         parent,
189.         start,
190.         goal);
191.
192.     panelGraph_1003.setVisited(visited);
193. }
194.
195. private void tampilkanHasil_1003(
196.     String metode,
197.     Set<String> visited,
198.     Map<String,String> parent,
199.     String start,
200.     String goal){

```

```

201.
202.     ArrayList<String> path = new ArrayList<>();
203.
204.     String current = goal;
205.
206.     while(current != null){
207.
208.         path.add(current);
209.         current = parent.get(current);
210.     }
211.
212.     Collections.reverse(path);
213.
214.     hasil_1003.setText(
215.         "Metode : "+metode+
216.         "\n\nUrutan Node : "+
217.         visited+
218.         "\n\nPath : "+
219.         path+
220.         "\n\nJumlah Node Dieksplorasi : "+
221.         visited.size());
222. }
223.
224. private void resetGraph_1003(){
225.
226.     hasil_1003.setText("");
227.     panelGraph_1003.clearVisited();
228. }
229.
230. public static void main(String[] args) {
231.
232.     SwingUtilities.invokeLater(() ->
233.         new PetaBandara_2511531003()
234.             .setVisible(true));
235. }
236.
237. class GraphPanel_1003 extends JPanel {
238.
239.     private Set<String> visited_1003 =
240.         new HashSet<>();
241.
242.     private Map<String,Point> posisi =
243.         new HashMap<>();
244.
245.     public GraphPanel_1003(){
246.
247.         posisi.put("Pintu Masuk",
248.             new Point(80,300));
249.
250.         posisi.put("Check In",
251.             new Point(220,200));
252.
253.         posisi.put("Security",
254.             new Point(380,130));
255.
256.         posisi.put("Ruang Tunggu",
257.             new Point(380,280));
258.
259.         posisi.put("Terminal A",
260.             new Point(520,120));
261.
262.         posisi.put("Terminal B",
263.             new Point(520,300));
264.
265.         posisi.put("Gate 1",
266.             new Point(650,80));
267.
268.         posisi.put("Gate 2",
269.             new Point(650,220));

```

```

270.
271.         posisi.put("Gate 3",
272.             new Point(650,380));
273.
274.         posisi.put("Bagasi",
275.             new Point(250,450));
276.     }
277.
278.     public void setVisited(
279.         Set<String> visited){
280.
281.         visited_1003 = visited;
282.         repaint();
283.     }
284.
285.     public void clearVisited(){
286.
287.         visited_1003.clear();
288.         repaint();
289.     }
290.
291.     protected void paintComponent(Graphics g){
292.
293.         super.paintComponent(g);
294.
295.         Graphics2D g2 =
296.             (Graphics2D) g;
297.
298.         for(String node :
299.             graph_1003.keySet()){
300.
301.             Point p1 = posisi.get(node);
302.
303.             for(String next :
304.                 graph_1003.get(node)){
305.
306.                 Point p2 = posisi.get(next);
307.
308.                 g2.drawLine(
309.                     p1.x,p1.y,
310.                     p2.x,p2.y);
311.             }
312.         }
313.
314.         for(String node :
315.             posisi.keySet()){
316.
317.             Point p = posisi.get(node);
318.
319.             if(visited_1003.contains(node))
320.                 g2.setColor(Color.GREEN);
321.             else
322.                 g2.setColor(Color.LIGHT_GRAY);
323.
324.             g2.fillOval(
325.                 p.x-25,
326.                 p.y-25,
327.                 50,
328.                 50);
329.
330.             g2.setColor(Color.BLACK);
331.
332.             g2.drawOval(
333.                 p.x-25,
334.                 p.y-25,
335.                 50,
336.                 50);
337.
338.             g2.drawString(

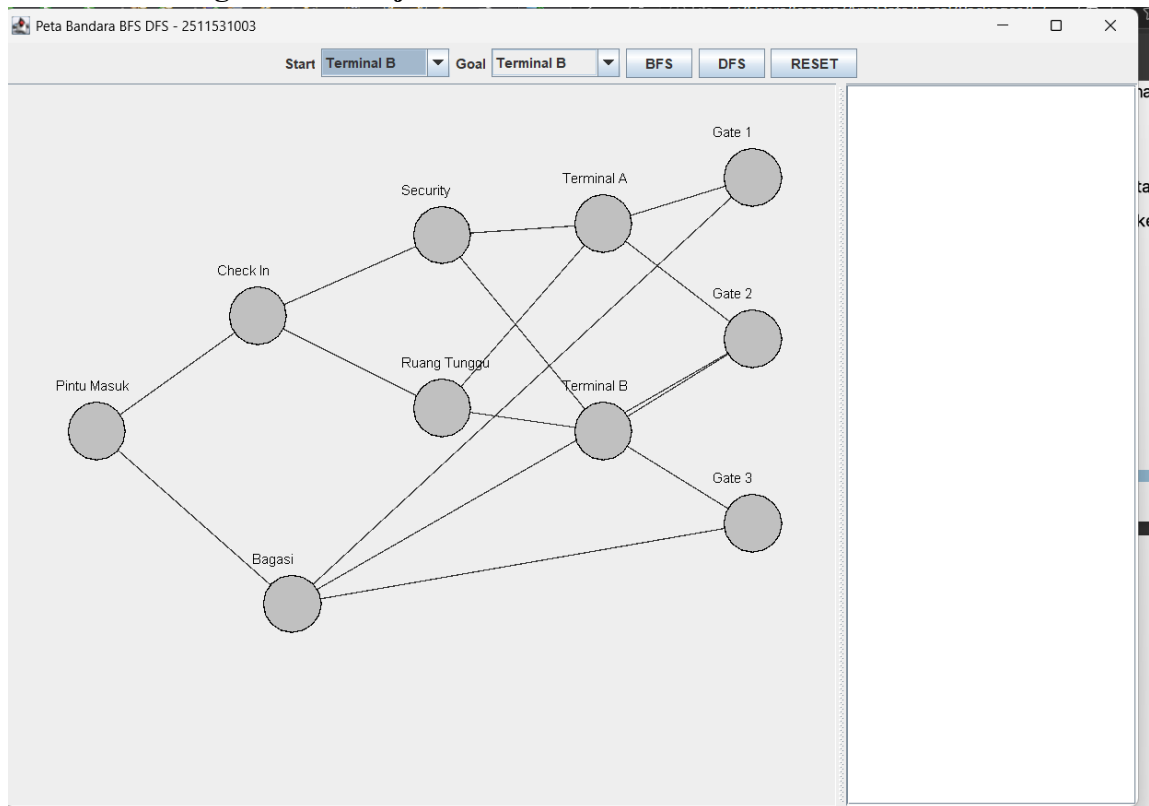
```

```

339.         node,
340.         p.x-35,
341.         p.y-35);
342.     }
343. }
344. }
345. }

```

2. Screenshot Program Saat Dijalankan



3. Laporan Singkat

a. Deskripsi peta yang dibuat

Program GUI ini menggunakan representasi graph untuk menggambarkan peta bandara. Setiap lokasi di bandara direpresentasikan sebagai node (simpul) dan setiap jalur penghubung antar lokasi direpresentasikan sebagai edge (sisi). Program menyediakan fitur pencarian jalur menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS). Pengguna dapat memilih Lokasi awal (start) dan Lokasi tujuan (Goal) kemudian pengguna memilih untuk menjalankan BFS atau DFS.

Pada GUI tersebut terdapat beberapa pilihan Lokasi pada peta bandara yaitu :

1. Pintu masuk
2. Check In
3. Security
4. Ruang Tunggu
5. Terminal A

6. Terminal B
7. Gate 1
8. Gate 2
9. Gate 3
10. Bagasi

b. Jumlah Node Dan Edge

Pada program tersebut terdapat **10 Node** yaitu :

1. Pintu Masuk
2. Check In
3. Security
4. Ruang Tunggu
5. Terminal A
6. Terminal B
7. Gate 1
8. Gate 2
9. Gate 3
10. Bagasi

Dan terdapat **15 Edge** yaitu :

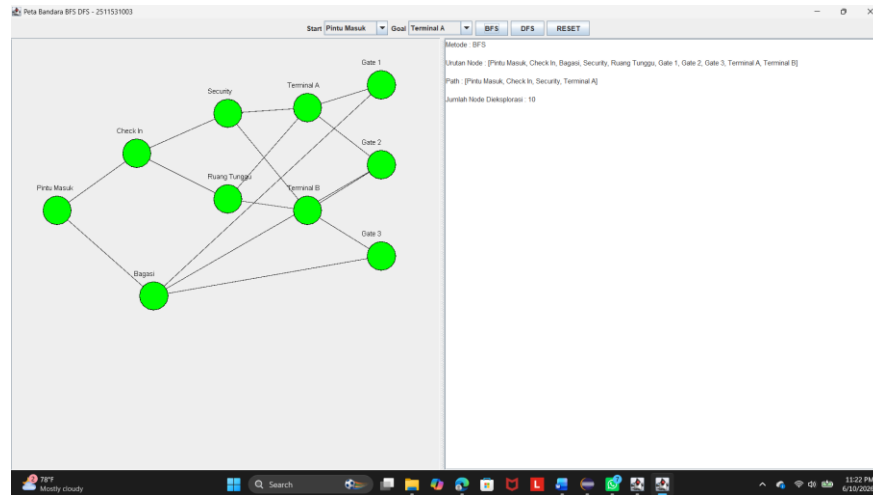
1. Pintu Masuk – Check In
2. Pintu Masuk – Bagasi
3. Check In – Security
4. Check In – Ruang Tunggu
5. Security – Terminal A
6. Security – Terminal B
7. Ruang Tunggu – Terminal A
8. Ruang Tunggu – Terminal B
9. Terminal A – Gate 1
10. Terminal A – Gate 2
11. Terminal B – Gate 2
12. Terminal B – Gate 3
13. Gate 1 – Bagasi
14. Gate 2 – Bagasi
15. Gate 3 – Bagasi

c. Hasil BFS.

Contoh :

Start : Pintu masuk

Goal : Terminal A

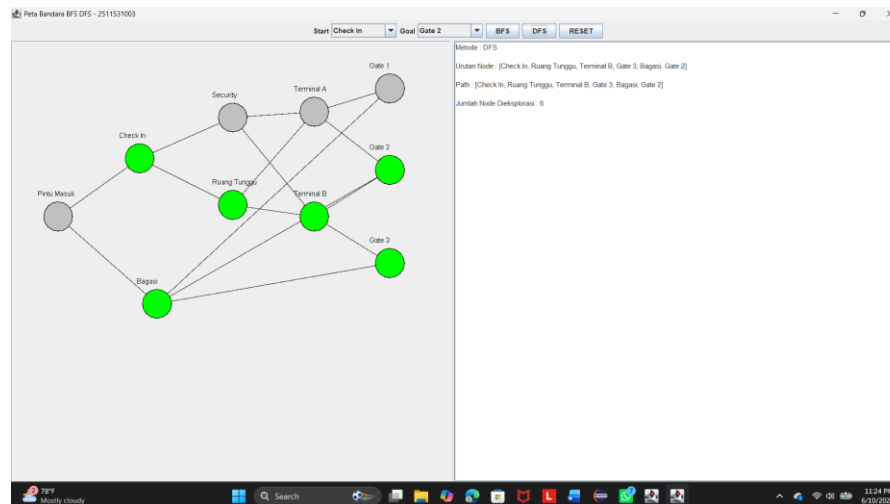


d. Hasil DFS

Contoh :

Start : Check In

Goal : Gate 2



e. Kesimpulan Perbandingan BFS dan DFS

FS menelusuri graph berdasarkan level sehingga mampu menemukan jalur terpendek, tetapi biasanya mengunjungi lebih banyak node dan membutuhkan memori yang lebih besar. Sementara itu, DFS menelusuri graph hingga ke cabang terdalam terlebih dahulu sehingga jumlah node yang dieksplorasi dapat lebih sedikit, namun jalur yang ditemukan belum tentu merupakan jalur terpendek. Oleh karena itu, BFS lebih cocok

digunakan untuk pencarian rute terpendek, sedangkan DFS lebih cocok digunakan untuk eksplorasi struktur graph secara mendalam.